

Bölüm 0 — Identity Kavramını Sıfırdan Anlamak

Linux Identity, ITDR ve identity security konularına geçmeden önce **identity kavramının kendisini** anlamamız gerekir.

Çünkü ileride göreceğimiz `UID`, `GID`, `PAM`, `SSH`, `Kerberos`, `SSSD`, `sudo`, `capabilities`, `auditd` gibi kavramların tamamı bir şekilde identity, authentication, authorization veya privilege kavramlarına bağlanır.

Bu nedenle önce temel kavramları birbirinden ayıracağız.

En önemli kural:

`Identity ≠ User ≠ Account ≠ Credential ≠ Session ≠ Permission ≠ Privilege`

0.1 Identity Nedir?

Identity, bir sistem içerisindeki bir varlığı diğerlerinden ayırt etmek ve o varlık hakkında bilgi tutmak için kullanılan kimlik kavramıdır.

En basit hâliyle:

Identity = "Bu varlık kim?" sorusunun sistem açısından cevabıdır.

Örneğin bir şirket ortamında:

Alice
Bob
Ahmet
Mehmet

Web Server
Database Server
Backup Service

bunların her biri sistem açısından farklı identity'lere sahip olabilir.

Buradaki önemli nokta şudur:

Identity yalnızca insanlara ait değildir.

Bir identity şunlara ait olabilir:

Human
Service
Machine
Workload

Örneğin:

Varlık	Identity örneği
İnsan	Alice
Servis	PostgreSQL
Makine	server-01
Workload	payment-service

Dolayısıyla identity kavramını yalnızca "kullanıcı" olarak düşünmek hatalıdır.

Şimdilik şu modeli kullanabiliriz:

Identity
├ Human
├ Service
├ Machine
└ Workload

ITDR açısından identity kavramının önemli olmasının nedeni de budur.

Bir identity hakkında şu soruları sorabiliriz:

Kim?
↓
Hangi identity?
↓
Hangi credential?
↓
Hangi session?
↓
Hangi privilege?
↓
Ne yaptı?

İlerleyen bölümlerde bu soruların her biri ayrı ayrı önem kazanacaktır.

0.2 Identifier Nedir?

Identifier, bir identity'yi tanımlamak veya diğer identity'lerden ayırt etmek için kullanılan değerdir.

Basitçe:

Identifier = "Bu identity'yi nasıl tanıyoruz?"

Örneğin Alice için:

```
Identity:
Alice

Identifier'lar:
UID      → 1000
Employee ID → 48021
E-mail   → alice@example.com
```

Buradaki değerler farklı sistemlerde Alice'i tanımlamak için kullanılabilir.

Linux açısından:

```
Alice
  ↓
UID 1000
```

Burada:

```
Identity   = Alice
Identifier = UID 1000
```

Dolayısıyla:

UID, identity'nin kendisi değildir.

UID, Linux'un Alice'i sayısal olarak tanımlamak için kullandığı identifier'lardan biridir.

Bu ayrım ileride çok önemli olacaktır.

0.3 Principal Nedir?

Principal, bir güvenlik sistemi tarafından tanınan ve kendisine kimlik veya yetki bağlamı atanabilen taraftır.

Daha basit bir ifadeyle:

Principal = Güvenlik sisteminin "bir taraf" olarak tanıdığı varlık.

Örneğin:

```
Alice
Bob
root
postgres
server-01
```

bir güvenlik sisteminde principal olabilir.

Örneğin:

```
Principal: Alice
```

veya:

```
Principal: postgres
```

olabilir.

Principal kavramı özellikle **authentication** ve **authorization** sistemlerinde önemlidir.

Örneğin:

```
Principal
  ↓
Alice
  ↓
Action: read
  ↓
Resource: report.pdf
```

Authorization sistemi daha sonra şunu değerlendirebilir:

```
Alice
  ↓
report.pdf
  ↓
READ
```

↓
ALLOW / DENY

Principal = Permission mı?

Hayır.

Burada önemli bir ayrım vardır.

Principal, **yetkinin kendisi değildir.**

Principal, yetkinin uygulanabileceği veya bağlanabileceği **taraftır.**

Örneğin:

```
Principal: Alice  
Permission: read  
Resource: report.pdf
```

Burada Alice principal'dır.

`read` ise bir permission'dır.

0.4 Subject Nedir?

Subject, güvenlik bağlamında bir işlem gerçekleştiren güvenlik öznesidir.

Basitçe:

Subject = Güvenlik açısından eylemi gerçekleştiren özne.

Örneğin:

```
Alice → reads → report.pdf
```

Burada Alice subject olarak düşünülebilir.

Fakat işletim sistemlerinde konu daha ilginç hâle gelir.

Alice bir process çalıştırabilir:

```
Alice  
↓  
bash
```

```
↓  
curl  
↓  
Network connection
```

Bu noktada kernel açısından işlemi gerçekleştiren varlık doğrudan "Alice" şeklinde düşünülmeyebilir.

İşlem, Alice'in credentials'ı ile çalışan bir **process** tarafından gerçekleştiriliyor olabilir.

Bu yüzden güvenlik modelini şu şekilde düşünebiliriz:

```
Subject  
↓  
Action  
↓  
Object
```

Örneğin:

```
Alice'in process'i  
↓  
read  
↓  
/etc/passwd
```

Buradaki temel fikir:

Subject, sistem açısından eylemi gerçekleştiren güvenlik öznesidir.

Linux'a geçtiğimizde subject kavramını process credentials, UID, GID, capabilities ve diğer güvenlik mekanizmalarıyla birlikte inceleyeceğiz.

0.5 Actor Nedir?

Actor, bir eylemi gerçekleştiren taraftır.

En basit tanımı:

Actor = Eylemi yapan taraf.

Örneğin:

```
Alice → deletes → test.txt
```

Burada Alice actor'dır.

Fakat eylemi bir process gerçekleştiriyorsa:

```
Malware Process
  ↓
modify
  ↓
/etc/passwd
```

burada process actor olarak düşünülebilir.

Daha karmaşık bir saldırı zincirinde birden fazla actor görebiliriz:

```
Attacker
  ↓
Stolen Credential
  ↓
Alice Account
  ↓
Shell Process
  ↓
sudo
  ↓
/etc/shadow
```

Burada farklı seviyelerde:

```
Attacker
Account
Process
```

gibi farklı actor'lar veya aktör bağlamları bulunabilir.

Bu nedenle:

Actor = Identity değildir.

Bir actor insan olabilir, process olabilir, servis olabilir veya başka bir sistem varlığı olabilir.

0.6 Account Nedir?

Account, bir sistem içerisinde bir identity için oluşturulan ve yönetilen kayıttır.

Linux'ta örneğin:

```
/etc/passwd
```

dosyasında şöyle bir kayıt bulunabilir:

```
alice:x:1000:1000:Alice:/home/alice:/bin/bash
```

Bu kayıt Alice için oluşturulmuş bir **account**'u temsil eder.

Dolayısıyla kabaca:

```
Identity
  ↓
Account
  ↓
Credentials
  ↓
Permissions / Attributes
```

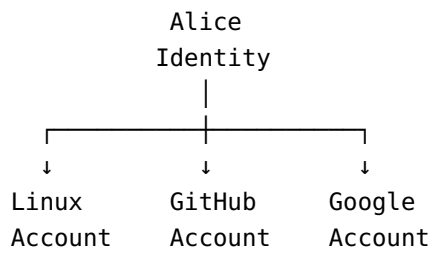
şeklinde düşünebiliriz.

Ancak:

Account ≠ Identity

Account, identity'nin belirli bir sistem içerisindeki yönetilebilir temsilidir.

Aynı kişi farklı sistemlerde farklı account'lara sahip olabilir:



Burada Alice tek bir insan identity'si iken farklı sistemlerde farklı account'larla temsil edilebilir.

0.7 User Nedir?

User, genellikle sistemi kullanan insanı ifade eder.

Örneğin:

Alice
Bob
Ahmet
Mehmet
Ayşe

birer user olabilir.

Fakat burada önemli bir tuzak vardır:

Her identity user değildir.

Örneğin Linux'ta:

postgres
www-data
nginx
backup

gibi account'lar insan kullanıcı olmayabilir.

Bunlar genellikle servis veya sistem amaçlı identity/account'lar olarak kullanılır.

Bu nedenle:

Human User
↓
Identity

doğrudur.

Fakat:

Identity
↓
Her zaman User

yanlıştır.

Daha doğru model:

Identity
├─ Human
├─ Service

└ Machine
└ Workload

Kısaca:

Her user bir identity'dir; fakat her identity bir user değildir.

0.8 Credential Nedir?

Burada çok önemli bir ayrım başlıyor.

Credential, bir identity'nin kendisini doğrulamak veya o identity adına authentication gerçekleştirmek için kullanılan bilgidir.

Örnekler:

Password
SSH Private Key
Kerberos Ticket
Access Token
Certificate
MFA Credential

Örneğin:

Identity:
Alice

Credential:
Password

veya:

Identity:
Alice

Credential:
SSH Private Key

şeklinde düşünebiliriz.

Fakat:

Password = Alice değildir.

Password, Alice'in identity'sini doğrulamak için kullanılan bir credential'dır.

Dolayısıyla:

Identity $\neq$ Credential

Bu ayrım güvenlik açısından çok önemlidir.

Örneğin saldırgan Alice'in password'ünü ele geçirirse:

```
Attacker
  ↓
Stolen Credential
  ↓
Authentication
  ↓
Alice Account
  ↓
Session
```

oluşabilir.

Burada saldırgan Alice'in identity'sini "çalmaz".

Alice'in identity'sini kullanmasına olanak sağlayan credential'ı ele geçirir.

Bu ayrım identity security açısından son derece önemlidir.

0.9 Authentication Nedir?

Authentication, bir identity'nin gerçekten iddia ettiği identity olduğunu doğrulama sürecidir.

En klasik soru:

"Sen kimsin?"

Örneğin:

```
Username: alice
Password: *****
```

Sistem credential'ı doğrular.

Başarılı olursa:

```
Claimed Identity
  +
  Credential
    ↓
  Authentication
    ↓
  Identity verified
```

şeklinde düşünebiliriz.

Önemli nokta:

Authentication, temel olarak "Kimsin?" sorusunu cevaplar.

Şu soruyu cevaplamaz:

"Ne yapabilirsin?"

Bu ikinci soru **authorization** ile ilgilidir.

0.10 Authorization Nedir?

Authorization, doğrulanmış bir identity'nin hangi işlemleri yapmasına izin verildiğini belirleme sürecidir.

Temel soru:

"Ne yapmana izin var?"

Örneğin:

```
Alice
  ↓
Authentication ✓
  ↓
Authorization
  ↓
Can Alice read /home/bob/file.txt?
```

Sonuç:

ALLOW

veya:

DENY

olabilir.

En önemli ayrım:

Authentication = Kimsin?

Authorization = Ne yapabilirsin?

Bu iki kavramı birbirine karıştırmamak gerekir.

0.11 Privilege Nedir?

Privilege, bir identity'nin veya process'in sahip olduğu özel yetki kapasitesidir.

Örneğin:

Normal User

ile:

root

aynı privilege seviyesine sahip değildir.

Örneğin:

Mahmut
↓
Normal Privilege

daha sonra:

Mahmut
↓
sudo
↓
Root Privilege

hâline gelebilir.

Buradaki önemli nokta:

Privilege değiştiğinde identity'nin mutlaka değişmesi gerekmez.

Mahmut hâlâ Mahmut olabilir.

Fakat process'in etkin yetki bağlamı değişmiş olabilir.

Bu nedenle:

Identity = Kim?

Privilege = Hangi özel yetki kapasitesine sahip?

şeklinde düşünmek faydalıdır.

0.12 Permission Nedir?

Permission, belirli bir resource üzerinde belirli bir işlemin yapılıp yapılamayacağını belirleyen izin kuralıdır.

Linux'taki klasik örnek:

```
r = read
w = write
x = execute
```

Örneğin:

```
Alice
↓
read
↓
report.txt
```

izinli olabilir.

Ama:

```
Alice
↓
write
```

↓
report.txt

izinli olmayabilir.

Dolayısıyla:

Permission
↓
Belirli bir resource
+
Belirli bir action
↓
ALLOW / DENY

şeklinde düşünebiliriz.

0.13 Privilege ile Permission Arasındaki Fark

Bu iki kavramın karıştırılması çok yaygındır.

Privilege

Bir identity'nin veya process'in sahip olduğu **özel yetki kapasitesidir**.

Permission

Belirli bir resource üzerinde belirli bir action'ın yapılıp yapılamayacağını belirleyen **izin kuralıdır**.

Örneğin:

Mahmut
↓
sudo privilege
↓
root-level operation

burada privilege vardır.

Ancak:

Mahmut
↓
write

↓
/etc/passwd

burada belirli resource üzerindeki permission değerlendirilir.

Bunu şöyle düşünebiliriz:

Privilege
↓
"Ne kadar / hangi seviyede yetki kapasitem var?"

Permission
↓
"Bu resource üzerinde şu işlemi yapabilir miyim?"

Permission, privilege'ın kendisi değildir.

Ayrıca modern Linux'ta gerçek authorization kararı yalnızca klasik Unix permission bitlerinden oluşmaz. ACL'ler, capabilities, LSM'ler, namespaces ve uygulamaya özgü policy mekanizmaları da karar sürecine dahil olabilir.

Bu nedenle:

Permission = Linux'taki rwx bitleri

demek de fazla basitleştirilmiş olur.

0.14 Authorization ile Permission Arasındaki Fark

Bu ikisi de sıkça karıştırılır.

Authorization, daha geniş bir kavramdır.

Authorization:

"Bu subject/principal bu action'ı bu resource üzerinde gerçekleştirebilir mi?"

sorusunun değerlendirilmesidir.

Permission ise bu kararı destekleyen **izin kurallarından biri** olabilir.

Örneğin:

Principal: Alice
Action: read
Resource: report.pdf

Authorization sistemi şu sonuca ulaşabilir:

ALLOW

Bu kararın arkasında:

Unix permissions
ACL
Application policy
Role
Group membership
Capability
Policy engine

gibi mekanizmalar bulunabilir.

Dolayısıyla:

Authorization
|
├── Permission
├── ACL
├── Role
├── Group
├── Policy
└── Capability

gibi daha geniş bir karar yapısı düşünülebilir.

Authorization bir karar/değerlendirme sürecidir. Permission ise bu süreçte kullanılan izin mekanizmalarından biridir.

Bu ayrım ileride Linux'a geçtiğimizde çok daha netleşecektir.

0.15 Session Nedir?

Session, bir identity'nin authentication sonrasında sistemle gerçekleştirdiği etkileşim bağlamıdır.

Örneğin Mahmut SSH ile sisteme bağlansın:

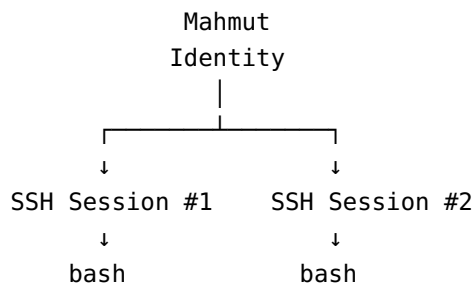
Mahmut
↓
SSH Authentication
↓
SSH Session
↓
Shell
↓
Commands

Bu session içerisinde örneğin:

- authentication context
- process'ler
- terminal/PTY
- environment
- privilege context
- session'a ilişkin diğer bilgiler

bulunabilir.

Aynı identity aynı anda birden fazla session'a sahip olabilir:



Dolayısıyla:

Identity ≠ Session

Identity "kim" olduğunu ifade eder.

Session ise o identity'nin belirli bir etkileşim bağlamını ifade eder.

0.16 Identity → Credential → Authentication → Session

Şimdi bu dört kavramı tek bir örnekte görelim.

```
Identity
Alice
|
▼
Credential
SSH Private Key
|
▼
Authentication
"Bu gerçekten Alice mi?"
|
▼
Session
SSH Session
```

Burada:

```
Alice = Identity

SSH Private Key = Credential

Authentication = Doğrulama süreci

SSH Session = Etkileşim bağlamı
```

Bir saldırgan Alice'in SSH private key'ini ele geçirirse:

```
Attacker
|
▼
Stolen Credential
|
▼
Authentication
|
▼
Alice's Identity
|
▼
New Session
```

oluşabilir.

İşte bu zincir ileride ITDR açısından çok önemli olacaktır.

0.17 Identity → Action → Resource

Şimdi identity'nin neden tek başına yeterli olmadığını görelim.

Bir sistemde sadece:

Alice

bilmek yeterli değildir.

Şunu da bilmemiz gerekir:

```
Alice
  ↓
hangi session?
  ↓
hangi process?
  ↓
hangi privilege?
  ↓
hangi action?
  ↓
hangi resource?
```

Örneğin:

```
Alice
  ↓
SSH Session #12
  ↓
bash
  ↓
sudo
  ↓
read
  ↓
/etc/shadow
```

Bu olayın güvenlik açısından anlamı yalnızca:

"Alice /etc/shadow'a erişti."

değildir.

Aynı zamanda:

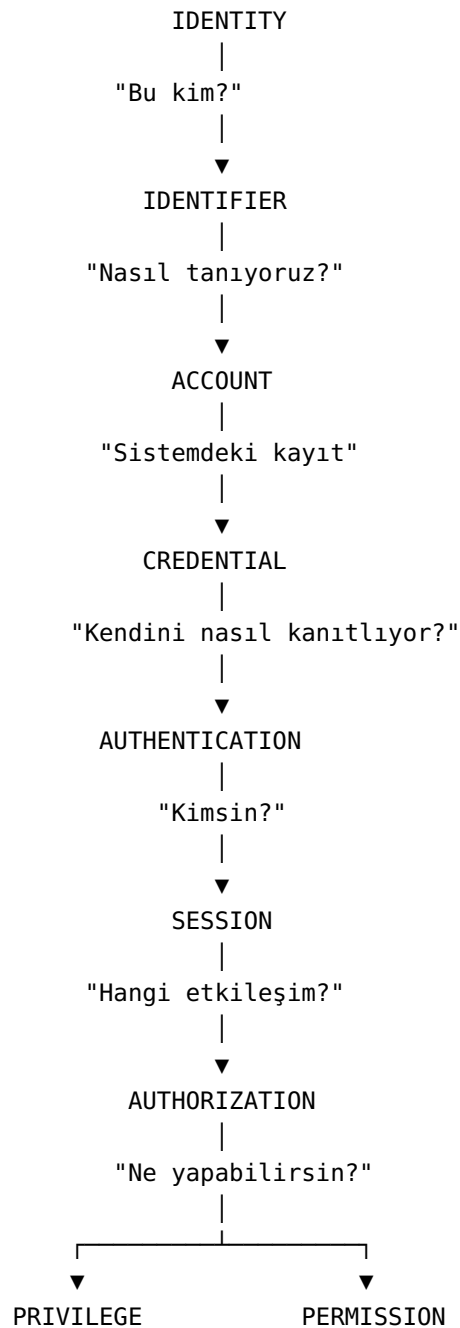
"Alice'in kimliği üzerinden oluşturulmuş belirli bir session içerisinde çalışan bir process, privilege elevation sonrasında hassas bir resource'a erişti."

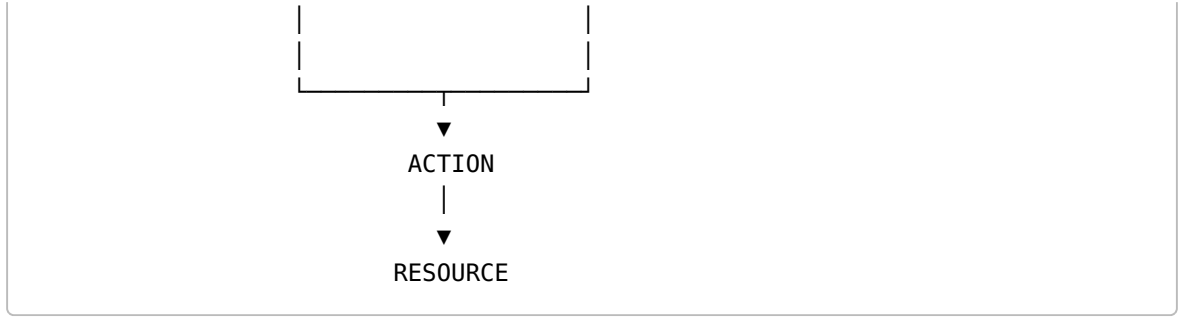
şeklinde değerlendirilmesi gerekir.

Bu yaklaşım ITDR'nin temel düşünce biçimine yakındır.

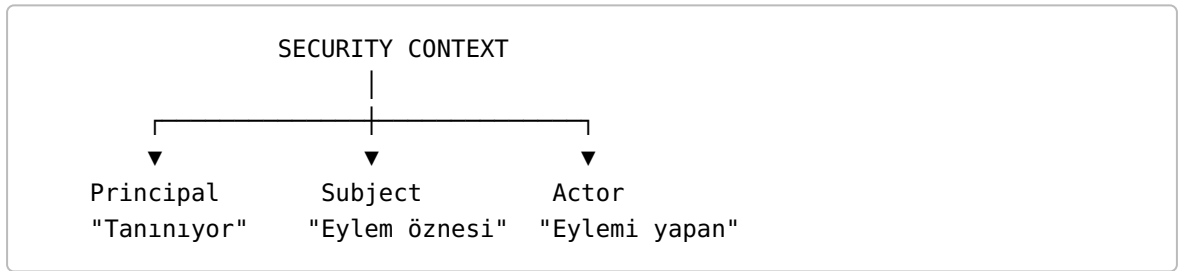
0.18 Bütün Kavramların Birbirine Bağlanması

Şimdi bütün kavramları tek bir model üzerinde görelim:





Burada **Principal**, **Subject** ve **Actor** biraz farklı bir eksenle düşünülmelidir:



Bu üç kavram birçok sistemde örtüşebilir; fakat kavramsal olarak aynı şey değildir.

0.19 En Çok Karıştırılan Kavramlar

Identity vs User

Identity = Sistem açısından tanımlanan varlık
User = Genellikle insan kullanıcı

Her user bir identity olabilir.

Fakat her identity user değildir.

Alice → User + Identity
postgres → Identity, fakat insan user değil
server-01 → Identity, fakat insan user değil

Identity vs Account

Identity = Kimlik
Account = Bu identity'nin belirli sistemdeki kaydı

Bir identity farklı sistemlerde farklı account'lara sahip olabilir.

Identity vs Credential

Identity = Kim?
Credential = Kim olduğunu nasıl kanıtlıyor?

Alice ≠ Alice's password
Alice ≠ Alice's SSH private key

Authentication vs Authorization

Authentication = Kimsin?
Authorization = Ne yapabilirsin?

Örneğin:

Password doğru
↓
Authentication ✓

Ama:

/etc/shadow'a erişim
↓
Authorization ✗

Authentication başarılı olabilirken authorization başarısız olabilir.

Privilege vs Permission

Privilege
"Özel yetki kapasitem nedir?"

Permission
"Bu resource üzerinde bu action'a izin var mı?"

Bunlar ilişkili fakat aynı değildir.

Authorization vs Permission

Authorization



Yetki kararının verilmesi/değerlendirilmesi

Permission



Bu kararı etkileyen belirli izin kuralı/mekanizması

Örneğin Linux'ta authorization kararı klasik Unix permissions'ın yanı sıra ACL, capabilities, LSM policy ve başka mekanizmalar tarafından da etkilenebilir.

Identity vs Session

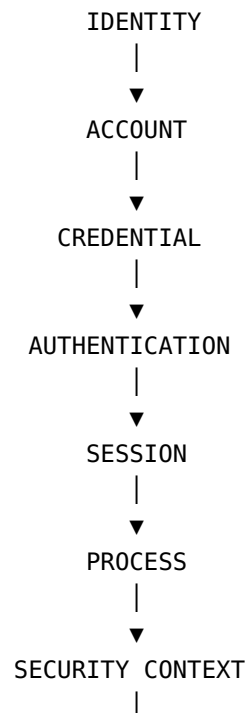
Identity = Kim?

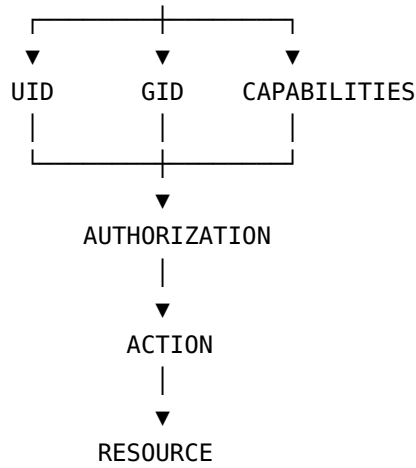
Session = Hangi etkileşim bağlamında?

Aynı identity birden fazla session'a sahip olabilir.

0.20 Linux'a Geçiş İçin İlk Güvenlik Modelimiz

Şimdilik Linux'u şu model üzerinden düşünelim:





Bu model ileride genişleyecek.

Örneğin daha sonra:

PAM
NSS
sudo
SSH
auditd
SELinux
AppArmor
SSSD
LDAP
Kerberos
AD

gibi bileşenleri bu modelin içerisine yerleştireceğiz.

0.21 ITDR Açısından Neden Önemli?

ITDR yalnızca:

"Bir kullanıcı ne yaptı?"

sorusuna bakmaz.

Daha kapsamlı olarak:

Hangi identity?
↓
Hangi account?

↓
Hangi credential?
↓
Hangi authentication?
↓
Hangi session?
↓
Hangi process?
↓
Hangi privilege?
↓
Hangi action?
↓
Hangi resource?

sorularını birbirine bağlamaya çalışır.

Örneğin:

Attacker
↓
Stolen SSH Key
↓
Alice Account
↓
SSH Authentication
↓
SSH Session
↓
Shell
↓
sudo
↓
Root-level privilege
↓
Read /etc/shadow

Bu olay tek başına bir process olayı değildir.

Aynı zamanda bir:

- identity olayı,
- credential olayı,
- authentication olayı,
- session olayı,
- privilege olayı,
- authorization olayı

olarak değerlendirilebilir.

İşte ITDR'nin EDR'den ayrıldığı düşünce alanlarından biri burada ortaya çıkar.

0.22 Bölüm 0 — Kısa Özet

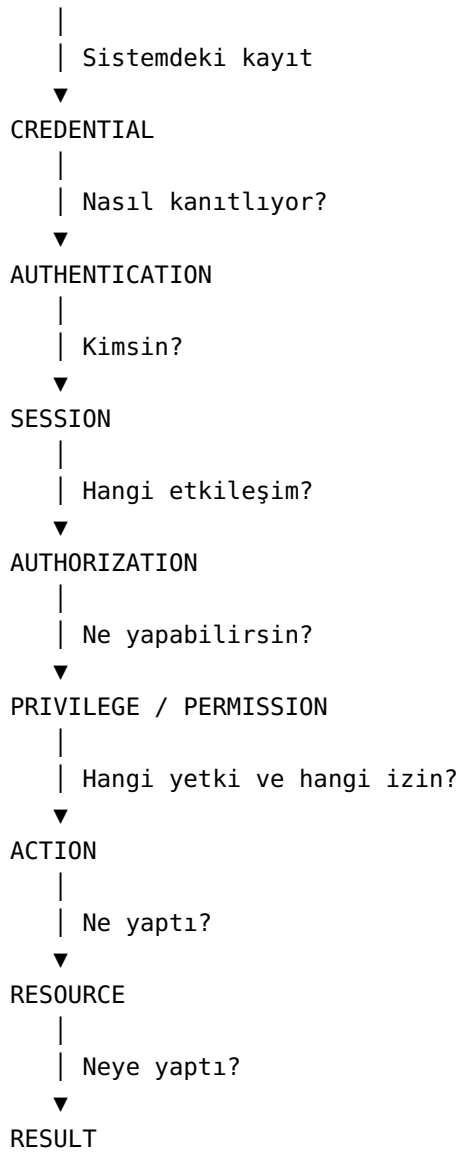
Bütün bölümü tek tabloya indirirsek:

Kavram	Temel soru	Kısa anlam
Identity	Bu kim?	Sistemde tanımlanan varlık
Identifier	Nasıl tanıyoruz?	Identity'yi tanımlayan değer
Principal	Güvenlik sistemi kimi tanıyor?	Güvenlik açısından tanınan taraf
Subject	Eylemin güvenlik öznesi kim?	Eylemi gerçekleştiren güvenlik öznesi
Actor	Eylemi kim yapıyor?	Eylemi gerçekleştiren taraf
Account	Sistemdeki kayıt hangisi?	Identity'nin sistemdeki yönetilebilir kaydı
User	Sistemi kullanan insan kim?	İnsan kullanıcı
Credential	Kendini nasıl kanıtlıyor?	Authentication için kullanılan kanıt
Authentication	Kimsin?	Identity doğrulama süreci
Session	Hangi etkileşim?	Identity'nin etkileşim bağlamı
Authorization	Ne yapabilirsin?	Yetki kararının/değerlendirmesinin yapılması
Privilege	Hangi özel yetki kapasitesine sahip?	Özel yetki seviyesi/kapasitesi
Permission	Bu resource üzerinde ne yapabilirsin?	Belirli action/resource için izin kuralı

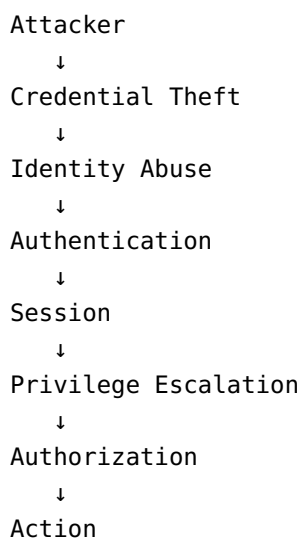
0.23 Akılda Tutulması Gereken En Önemli Formül

Şimdilik aşağıdaki zinciri ezberlemek yerine **anlamını kavramak** yeterlidir:

```
IDENTITY
|
| Kim?
▼
IDENTIFIER
|
| Nasıl tanıyoruz?
▼
ACCOUNT
```



Ve saldırgan perspektifinden:



↓
Resource

Bu zincir, ileride Linux Identity Threat Detection & Response tasarlarken kullanacağımız temel düşünce modelinin başlangıç noktasıdır.

Bir identity'yi anlamak, yalnızca "kullanıcı kim?" demek değildir.

Asıl soru şudur:

"Bu varlık kim, nasıl tanınıyor, kendisini nasıl doğruluyor, hangi session üzerinden hareket ediyor, hangi yetki bağlamına sahip ve hangi resource üzerinde ne yapıyor?"